```
// 09-anotacoes.js — Sistema de anotações (texto rico, tabelas)
// Parte do script do app (v2.0.1). Arquivo comum, carregado em ordem pelo index.
// --- SISTEMA DE ANOTAÇÕES (múltiplas notas, texto rico, tabelas) ---

function stripHtmlAnotacao(html) {
    const tmp = document.createElement('div');
    tmp.innerHTML = html || '';
    // textContent concatena blocos (<p>, <div>, <li>...) sem nenhum espaço
    // entre eles — "RIOCLARENSE Ped. 1698" + "Pedido: SERINGA..." virava
    // "RIOCLARENSE Ped. 1698Pedido: SERINGA..." colado. Insere um espaço no
    // fim de cada bloco antes de extrair o texto, sem criar quebra de linha.
    tmp.querySelectorAll('p, div, li, h1, h2, h3, h4, br').forEach(el => {
        el.insertAdjacentText('afterend', ' ');
    });
    return (tmp.textContent || tmp.innerText || '').replace(/\s+/g, ' ').trim();
}

function renderListaAnotacoes() {
    const container = document.getElementById('lista-anotacoes-container');
    if (!container) return;

    const termo = filtroAnotacoesTexto.trim().toUpperCase();
    const lista = listaAnotacoes.filter(a => {
        if (!termo) return true;
        const titulo = (a.titulo || '').toUpperCase();
        const texto = stripHtmlAnotacao(a.conteudo).toUpperCase();
        return titulo.includes(termo) || texto.includes(termo);
    });

    if (lista.length === 0) {
        container.innerHTML = `<div class="empty-state">${listaAnotacoes.length =
        return;
    }

    container.innerHTML = lista.map(a => {
        const snippet = stripHtmlAnotacao(a.conteudo).slice(0, 90) || 'Sem conteú
        const data = a.atualizadoEm ? new Date(a.atualizadoEm).toLocaleString('pt
        const concluida = a.status === 'concluido';
        const sinalizada = !!a.sinalizada;
        const selecionada = selecaoAnotacoesAtiva && anotacoesSelecionadas.has(a.
        return `<div class="nota-item anotacao-item ${concluida ? 'anotacao-concl
            <div class="nota-detalhes">${snippet}${snippet.length >= 90 ? '…' : '
            <div class="nota-data">Atualizado em: ${data}</div>
```

```
            ${!selecaoAnotacoesAtiva ? `<div class="anotacao-card-actions">
                <button type="button" class="anotacao-icon-btn ${sinalizada ? 'is
                <button type="button" class="anotacao-icon-btn ${concluida ? 'is-
                <button type="button" class="anotacao-icon-btn is-delete" title="
            </div>` : ''}
        </div>`;
    }).join('');
}

function toggleFlagAnotacao(id) {
    const nota = listaAnotacoes.find(a => a.id === id);
    if (!nota) return;
    const novoValor = !nota.sinalizada;
    anotacoesTextoCollection.doc(id).update({ sinalizada: novoValor })
        .catch(e => { console.error('Erro ao sinalizar anotação:', e); toast('× E
}

function toggleConcluidaAnotacao(id) {
    const nota = listaAnotacoes.find(a => a.id === id);
    if (!nota) return;
    const novoStatus = nota.status === 'concluido' ? null : 'concluido';
    anotacoesTextoCollection.doc(id).update({ status: novoStatus })
        .then(() => toast(novoStatus ? '✓ Marcada como concluída.' : 'Reaberta.')
        .catch(e => { console.error('Erro ao concluir anotação:', e); toast('× Er
}

function excluirAnotacaoRapida(id) {
    const nota = listaAnotacoes.find(a => a.id === id);
    showConfirmModal({
        title: 'Excluir Anotação',
        message: `Deseja excluir "${nota ? (nota.titulo || 'Sem título') : 'esta
        confirmText: 'Excluir',
        confirmClass: 'danger',
        onConfirm: async () => {
            try {
                await anotacoesTextoCollection.doc(id).delete();
                toast('🗑 Anotação excluída.');
            } catch (e) {
                console.error('Erro ao excluir anotação:', e);
                toast('× Erro ao excluir.');
            }
        }
    });
}

function filtrarAnotacoes(texto) {
```

```
    filtroAnotacoesTexto = texto;
    renderListaAnotacoes();
}

// --- Seleção múltipla para exportar relatório consolidado pro WhatsApp
// (vários pedidos diferentes numa mensagem só, sem redigitar). ---
function toggleModoSelecaoAnotacoes() {
    selecaoAnotacoesAtiva = !selecaoAnotacoesAtiva;
    anotacoesSelecionadas.clear();
    const btn = document.getElementById('btn-selecionar-anotacoes');
    const barra = document.getElementById('anotacoes-bulk-bar');
    if (btn) btn.classList.toggle('active', selecaoAnotacoesAtiva);
    if (barra) barra.style.display = selecaoAnotacoesAtiva ? 'flex' : 'none';
    atualizarContadorSelecaoAnotacoes();
    renderListaAnotacoes();
}
function toggleSelecaoAnotacao(id) {
    if (anotacoesSelecionadas.has(id)) anotacoesSelecionadas.delete(id);
    else anotacoesSelecionadas.add(id);
    atualizarContadorSelecaoAnotacoes();
    renderListaAnotacoes();
}
function atualizarContadorSelecaoAnotacoes() {
    const el = document.getElementById('anotacoes-selecao-contador');
    if (el) el.textContent = anotacoesSelecionadas.size > 0 ? `${anotacoesSelecio
}
async function exportarAnotacoesWhatsapp() {
    if (anotacoesSelecionadas.size === 0) return toast('Selecione ao menos uma an
    const selecionadas = listaAnotacoes.filter(a => anotacoesSelecionadas.has(a.i
    // Mantém a ordem em que aparecem na lista (mais recentes primeiro), não a
    // ordem de clique.
    const blocos = selecionadas.map(a => {
        const texto = stripHtmlAnotacao(a.conteudo).trim();
        return `*${a.titulo || 'Sem título'}*${texto ? '\n' + texto : ''}`;
    });
    const relatorio = blocos.join('\n\n———\n\n');
    if (navigator.share) {
        try {
            await navigator.share({ title: 'Relatório de Anotações', text: relato
        } catch (e) {
            // usuário cancelou o share sheet — não é erro
        }
    } else {
        await navigator.clipboard.writeText(relatorio);
        toast('✓ Relatório copiado! Cole no WhatsApp.');
    }
```

```
}

function abrirNovaAnotacao() {
    voltarAnotacaoEditorParaCentral = null; // reseta — só a Central religa essa
    anotacaoAtualId = null;
    document.getElementById('anotacao-titulo').value = '';
    document.getElementById('anotacao-corpo').innerHTML = '';
    aplicarEstadoEspacamentoEditor('', false);
    switchToScreen('screen-anotacoes-editor', 'Nova Anotação');
    setTimeout(() => { document.getElementById('anotacao-titulo').focus(); atuali
}

function abrirAnotacao(id, viaCentral) {
    const nota = listaAnotacoes.find(a => a.id === id);
    if (!nota) return;
    if (!viaCentral) voltarAnotacaoEditorParaCentral = null; // reseta se aberta
    anotacaoAtualId = id;
    document.getElementById('anotacao-titulo').value = nota.titulo || '';
    document.getElementById('anotacao-corpo').innerHTML = nota.conteudo || '';
    aplicarEstadoEspacamentoEditor(nota.espacamentoLinha || '', !!nota.paragrafoC
    switchToScreen('screen-anotacoes-editor', nota.titulo || 'Anotação');
    setTimeout(atualizarEstadoToolbarAnotacao, 300);
}

// ============================================================
// CENTRAL DO PEDIDO — tela própria (não mais um painel dentro do editor de
// anotação). cotacoes/{pedido} continua sendo a única fonte da cotação;
// anotacoesTexto continua sendo a única fonte de ocorrências/divergências/
// status/histórico. Esta tela só JUNTA as duas na apresentação — nenhum
// dado é copiado de um lado pro outro.
//
// STATUS de uma divergência: 'pendente' (padrão, inclusive pra ocorrências
// antigas sem esse campo — nunca migramos histórico antigo só pra
// adicionar o campo), 'resolvida' (problema corrigido de fato) ou
// 'encerrada' (caso finalizado sem necessariamente ter sido corrigido).
// HISTÓRICO é append-only: uma mudança de status NUNCA apaga ou reescreve
// uma entrada anterior, só acrescenta uma nova. O texto que foi gerado e
// salvo no momento da identificação nunca muda retroativamente.
// ============================================================
let centralPedidoAtual = null;
let centralFornecedoresAbertos = new Set();
let centralStatusFormAberto = new Set();
let voltarAnotacaoEditorParaCentral = null; // guarda o pedido, se o editor de te

const ROTULOS_HISTORICO = { identificacao: 'Identificado', resolucao: 'Resolvido'
```

```
// Fase 37: a cotação individual tem abas (Produtos · Anotações · Relatório · Dad
function alternarAbaCentral(aba) {
    document.querySelectorAll('#screen-central-pedido .central-aba-pane').forEach
    document.querySelectorAll('#screen-central-pedido .central-abas .xml-filtro-b
}
function abrirCentralPedido(pedido) {
    if (!pedido) return;
    centralPedidoAtual = pedido;
    centralFornecedoresAbertos = new Set();
    centralStatusFormAberto = new Set();
    alternarAbaCentral('produtos'); // Fase 37: sempre abre na aba Produtos
    switchToScreen('screen-central-pedido', pedido);
    renderCentralPedidoCompleto(pedido);
}

// Linhas de texto (uma por material) de UMA divergência — reaproveita
// linhaMaterial()/paragrafoAvaria(), já usados na geração de texto da
// Auditoria (mesma lógica, não duplicada).
// Texto natural de uma ocorrência que NÃO é por material (multiMaterial:
// false) — hoje "fornecedor não entregou" e "frete em desacordo". Extraído
// como função própria pra não duplicar a mesma string em dois lugares
// (Central do Pedido via linhasDeMateriais, e a saída da Auditoria via
// gerarCorpoAuditoria) — um ajusta, o outro acompanha automaticamente.
function textoOcorrenciaUnica(tipo, c) {
    if (tipo === 'fornecedor_nao_entregou' || tipo === 'frete_indevido') return a
    return '';
}

// 2.10.5: auditorias ANTIGAS guardavam o produto em d.campos (um só) em vez de d
// e três divergências diferentes pareciam a mesma. Esta função devolve sempre a
function materiaisDaDivergencia(d) {
    if (!d) return [];
    if (Array.isArray(d.materiais) && d.materiais.length) return d.materiais;
    const def = TIPOS_DIVERGENCIA_AUDITORIA[d.tipo];
    const c = d.campos || {};
    const temProduto = c.produto || c.produtoPedido || c.produtoFaturado;
    if ((!def || def.multiMaterial) && temProduto) return [c];
    return [];
}
function linhasDeMateriais(d) {
    const def = TIPOS_DIVERGENCIA_AUDITORIA[d.tipo];
    if (!def) { // tipo que não existe mais: mostra o que foi gravado, sem invent
        return materiaisDaDivergencia(d).map(m => [m.produto, m.quantidadePedida
    }
    if (def.multiMaterial) return materiaisDaDivergencia(d).map(m => (d.tipo ===
    const texto = textoOcorrenciaUnica(d.tipo, d.campos || {});
```

```
    return texto ? [texto] : [];
}

// Bloco de UMA divergência: linhas de material + status + histórico +
// controle pra mudar o status. ocIdx/divIdx são os índices reais dentro de
// nota.ocorrencias[ocIdx].divergencias[divIdx] — é como alterarStatusDivergencia
// sabe exatamente o que atualizar no Firestore.
function renderBlocoDivergencia(d, oc, ocIdx, divIdx) {
    const status = d.status || 'pendente'; // ocorrências antigas sem status vira
    const linhas = linhasDeMateriais(d);
    if (linhas.length === 0) return '';
    const linhasHTML = linhas.map(l => `<div class="central-item-linha">${l.repla

    const historico = d.historico || [];
    const historicoHTML = historico.length ? `<div class="central-historico">
        ${historico.map(h => `<div class="central-historico-item">${formatarDataB
    </div>` : '';

    const key = `${ocIdx}-${divIdx}`;
    const formAberto = centralStatusFormAberto.has(key);
    const statusFormHTML = formAberto ? `
        <div class="central-status-form">
            <select id="central-status-select-${key}" class="form-field">
                <option value="pendente" ${status === 'pendente' ? 'selected' : '
                <option value="resolvida" ${status === 'resolvida' ? 'selected' :
                <option value="encerrada" ${status === 'encerrada' ? 'selected' :
            </select>
            <input type="text" id="central-status-obs-${key}" class="form-field"
            <button type="button" class="actions-button is-success" onclick="conf
        </div>` : '';

    return `<div class="central-divergencia-bloco">
        ${linhasHTML}
        <div class="central-divergencia-meta">
            <span class="central-status-badge status-${status}">${status === 'pen
            ${oc.notaFiscal ? `<span class="central-nf-tag">NF ${oc.notaFiscal}</
            <button type="button" class="central-status-toggle" onclick="toggleCe
        </div>
        ${historicoHTML}
        ${statusFormHTML}
    </div>`;
}

function renderCentralPedidoCompleto(pedido) {
    const cotacao = listaCotacoes.find(c => c.pedido === pedido);
    const nota = listaAnotacoes.find(a => a.pedido === pedido);
```

```
document.getElementById('central-titulo-pedido').textContent = cotacao && cot
// Fase 34: subtítulo com o essencial do pedido + resumo de entrega.
// Fase 37: pedido sem cotação não tem o que separar em abas — mostra tudo ju
document.getElementById('screen-central-pedido').classList.toggle('central-se
const subEl = document.getElementById('central-subtitulo-pedido');
if (subEl) {
    if (cotacao) {
        const nForn = (cotacao.fornecedores || []).length;
        subEl.innerHTML = `<div class="central-sub-linha">${nForn} fornecedor
        subEl.style.display = 'block';
    } else { subEl.innerHTML = ''; subEl.style.display = 'none'; }
}

const headerCard = document.getElementById('central-header-card');
const semCotacao = document.getElementById('central-sem-cotacao');
if (cotacao) {
    headerCard.style.display = 'block';
    semCotacao.style.display = 'none';
    document.getElementById('central-origem').value = cotacao.origem || '';
    document.getElementById('central-data-pedido').value = cotacao.dataPedido
    document.getElementById('central-data-limite').value = cotacao.dataLimite
    document.getElementById('central-observacao-cotacao').value = cotacao.obs
} else {
    headerCard.style.display = 'none';
    semCotacao.style.display = 'block';
}

// Todas as divergências de todas as ocorrências, achatadas, guardando os
// índices reais (ocIdx/divIdx) pra alteração de status saber onde mexer,
// e o id da anotação (pra Firestore) embutido em cada ocorrência.
const todasDivergencias = [];
(nota && nota.ocorrencias ? nota.ocorrencias : []).forEach((oc, ocIdx) => {
    const ocComId = { ...oc, _anotacaoId: nota.id };
    (oc.divergencias || []).forEach((d, divIdx) => todasDivergencias.push({ d
});

const container = document.getElementById('central-fornecedores-container');

if (cotacao) {
    const compItens = compararFontesPedido(pedido);
    const normNome = x => String(x || '').trim().toUpperCase();
    const fornecedoresHTML = (cotacao.fornecedores || []).map(f => {
        const nomeExibicao = nomeExibicaoFornecedor(f.razaoSocial, f.cnpj);
        const produtos = (cotacao.itens || []).filter(it => normalizarCnpj(it
        const divergenciasForn = todasDivergencias.filter(x => normalizarCnpj
```

```
const aberto = centralFornecedoresAbertos.has(f.cnpj);
const compForn = compItens.filter(item => normalizarCnpj(item.cnpjFor
const chegada = (cotacao.chegadasPorFornecedor || {})[normalizarCnpj(
// Fase 31: status de entrega CALCULADO (XML do app + ERP + itens); a
const entrega = calcularEntregaFornecedorPedido(pedido, f, compForn);
// 2.1.3: botões (chegou / abrir NF / associar NF) ficam FORA da part
const dadosEnt = dadosEntregaFornecedor(pedido, f, entrega);
// 2.9.3: fechado, o cartão mostra só o cabeçalho e a barra de ações;
const acoesLista = dadosEnt ? dadosEnt.acoes : [acaoAssociarNf(pedido
const acoesHTML = `<div class="ctl-bar cf-bar">${acoesLista.map(a =>
    ${htmlPainelAssociarNf(pedido, f.cnpj)}`;
const detalhesEntregaHTML = ((dadosEnt && dadosEnt.detalhes) || '') +

// Fase 27: cada divergência da auditoria vai pra linha do PRIMEIRO
// produto cujo nome bate EXATAMENTE (sem aproximação) com um dos
// materiais dela. Sem correspondência exata (ou tipo sem material,
// ex.: "fornecedor não entregou"), fica em "Outras divergências" —
// nenhuma divergência some nem aparece duplicada.
const divPorProduto = {};
const divSoltas = [];
divergenciasForn.forEach(x => {
    const nomes = materiaisDaDivergencia(x.d).map(m => normNome(m.pro
    const codsRef = materiaisDaDivergencia(x.d).map(m => m.codProduto
    const alvo = produtos.find(it => codsRef.includes(it.codProduto))
    if (alvo) (divPorProduto[alvo.codProduto] = divPorProduto[alvo.co
    else divSoltas.push(x);
});

const produtosHTML = produtos.length
    ? produtos.map(it => {
        const comp = compForn.find(c => c.codProduto === it.codProdut
        const divs = divPorProduto[it.codProduto] || [];
        return renderLinhaProduto(it, `${f.cnpj}-${it.codProduto}`, {
            comp, chegou: !!chegada,
            compHTML: comp ? renderComparacaoItemHTML(pedido, comp, t
            divsHTML: divs.map(x => renderBlocoDivergencia(x.d, x.oc,
            divsPendentes: divs.filter(x => (x.d.status || 'pendente'
        });
    }).join('')
    : '<div class="central-item-vazio">Nenhum produto cotado registra

const soltasHTML = divSoltas.length
    ? `<div class="central-secao-titulo">Outras divergências deste fo
    : '';

// 2.0.9: "com divergência" conta só o que está EM ABERTO (item sem a
```

```
    const nComDiv = compForn.filter(c => c.divergencias.length && !c.reso
    const nPend = divergenciasForn.filter(x => (x.d.status || 'pendente')
    const emErro = nComDiv > 0 || nPend > 0;
    const entregaOk = !!(entrega && entrega.estado === 'completa');
    const classeFornecedor = emErro ? 'st-erro' : (entregaOk ? 'st-ok' :
    const rot = entrega ? rotuloEntregaFornecedor(entrega) : null;
    const classeEntrega = rot ? (rot.classe === 'pronto' ? 'rs-ok' : rot.
    const partesResumo = [
        `<span>${produtos.length} item(ns)</span>`,
        nComDiv ? `<strong class="rs-erro">${nComDiv} com divergência em
        nPend ? `<strong class="rs-erro">${nPend} auditoria(s) pendente(s
    ].filter(Boolean);
    const resumo = partesResumo.join(' · ');

    // 2.9.0: cartão no padrão do app — iniciais, nome, status, itens/val
    const valorCot = (typeof valorTotalCotacaoPorFornecedor === 'function
    const stRot = emErro ? ['', 'Com pendência'] : (entregaOk ? ['ok', 'E
    const pendTxt = [nComDiv ? `${nComDiv} item(ns) com divergência em ab
    const faixaHTML = [dadosEnt && dadosEnt.curta, pendTxt].filter(Boolea
    return `<div class="central-fornecedor${aberto ? ' aberto' : ''} ${cl
        <div class="central-fornecedor-header" onclick="toggleCentralForn
            <div class="ctl-av">${iniciaisControle(nomeExibicao)}</div>
            <div class="cf-topo-info">
                <div class="cf-nome">${nomeExibicao}</div>
                ${f.cnpj ? `<div class="cf-cnpj">CNPJ ${formatarCnpjExibi
            </div>
            <span class="ctl-st ${emErro ? 'err ctl-link' : stRot[0]}"${e
            <i class="fa-solid fa-chevron-right"></i>
        </div>
        ${acoesHTML}
        <div class="central-colapso"><div class="central-colapso-inner"><
            ${faixaHTML}
            <div class="cf-tiles"><div><span>Itens</span><strong>${produt
            ${detalhesEntregaHTML ? `<div class="cf-detalhes">${detalhesE
            ${produtosHTML}
            ${soltasHTML}
        </div></div></div>
    </div>`;
}).join('');

const semFornecedor = todasDivergencias.filter(x => !x.oc.fornecedorCnpj)
const abertoSem = centralFornecedoresAbertos.has('_sem_cnpj');
const semFornecedorHTML = semFornecedor.length ? `<div class="central-for
    <div class="central-fornecedor-header" onclick="toggleCentralForneced
        <div class="cf-topo-info"><div class="cf-nome">Outras ocorrências
        <i class="fa-solid fa-chevron-right"></i>
```

```
            </div>
            <div class="central-colapso"><div class="central-colapso-inner"><div
                ${semFornecedor.map(x => `<div style="margin-bottom:6px;font-size
            </div></div></div>
        </div>` : '';

        container.innerHTML = htmlItensForaDoPedido(pedido) + fornecedoresHTML +
    } else if (nota) {
        // Sem cotação cadastrada, mas já existem ocorrências registradas (ex:
        // auditorias antigas de antes da cotação existir) — mostra tudo junto,
        // sem agrupamento por fornecedor (não há CNPJ pra agrupar por).
        container.innerHTML = todasDivergencias.length
            ? todasDivergencias.map(x => `<div style="margin-bottom:6px;font-size
            : '<div class="central-item-vazio">Nenhuma ocorrência registrada aind
    } else {
        container.innerHTML = '';
    }

    document.getElementById('central-btn-texto-livre').style.display = nota ? ''

    // Anotação embutida no próprio contexto do pedido (correção desta rodada):
    // mesma anotação/estrutura de sempre (anotacoesTextoCollection, casada
    // por `pedido`), só que exibida e editável aqui, sem precisar navegar pra
    // Anotações. Guarda contra sobrescrever o que a pessoa está digitando: só
    // substitui o conteúdo do campo se ele não estiver com foco no momento.
    centralAnotacaoId = nota ? nota.id : null;
    const anotCard = document.getElementById('central-anotacao-card');
    const anotVazia = document.getElementById('central-anotacao-vazia');
    if (anotCard && anotVazia) {
        if (nota) {
            anotCard.style.display = 'block';
            anotVazia.style.display = 'none';
            const campo = document.getElementById('central-anotacao-conteudo');
            if (campo && document.activeElement !== campo) campo.innerHTML = nota
        } else {
            anotCard.style.display = 'none';
            anotVazia.style.display = 'block';
        }
    }

}

let centralAnotacaoId = null;
let autoSaveAnotacaoInlineCentralTimeout = null;

function agendarSalvarAnotacaoInlineCentral() {
```

```
    clearTimeout(autoSaveAnotacaoInlineCentralTimeout);
    autoSaveAnotacaoInlineCentralTimeout = setTimeout(salvarAnotacaoInlineCentral
}

async function salvarAnotacaoInlineCentral() {
    if (!centralAnotacaoId) return;
    const campo = document.getElementById('central-anotacao-conteudo');
    if (!campo) return;
    try {
        await anotacoesTextoCollection.doc(centralAnotacaoId).update({ conteudo:
    } catch (e) {
        console.error('Erro ao salvar anotação inline da Central:', e);
    }
}

// Reaproveita a mesma criação usada na cotação (criarOuAtualizarAnotacaoDaCotaca
// — sem estrutura de dados nova. Preenche o campo na hora, sem esperar o
// listener em tempo real, pra já aparecer editável assim que criada.
async function criarAnotacaoInlineCentral() {
    if (!centralPedidoAtual) return;
    const cotacao = listaCotacoes.find(c => c.pedido === centralPedidoAtual);
    const origem = cotacao ? cotacao.origem : '';
    const dataPedido = cotacao ? cotacao.dataPedido : '';
    const dataLimite = cotacao ? cotacao.dataLimite : '';
    const fornecedores = cotacao ? cotacao.fornecedores : [];
    try {
        const id = await criarOuAtualizarAnotacaoDaCotacao(centralPedidoAtual, or
        if (!id) return toast('× Erro ao criar anotação.');
        centralAnotacaoId = id;
        document.getElementById('central-anotacao-card').style.display = 'block';
        document.getElementById('central-anotacao-vazia').style.display = 'none';
        document.getElementById('central-anotacao-conteudo').innerHTML = montarCa
    } catch (e) {
        console.error('Erro ao criar anotação do pedido:', e);
        toast('× Erro ao criar anotação.');
    }
}

// Produto sempre mostra o que realmente veio do XML — nunca inventa uma
// descrição. Quando o fornecedor não preencheu o campo de descrição (existe
// no XML real: alguns itens vêm com <Comentario></Comentario> vazio),
// mostra isso como ausência de dado, nunca como se fosse o nome do produto,
// e o código do fornecedor fica sempre visível (base pra associação futura
// de código externo → interno).
// Reconstrói a cotação com fidelidade total (regra definitiva combinada) —
// nunca usa observação/fabricante/código como substituto do nome oficial.
```

```
// Nome oficial só existe depois que o relatório do SmartCompras for colado
// e cruzado (código + CNPJ); até lá, fica marcado como pendente de
// complementação, nunca escondido nem inventado.
let centralProdutosExpandidos = new Set();
let centralAssociacaoBuscaAberta = new Set();

function renderAssociacaoSpDataWidget(it, chaveUnica) {
    if (!it.codProduto) return '';
    const chaveWidget = `assoc-${chaveUnica}`;
    const associacao = listaAssociacoesSpData.find(a => a.codigoSmartCompras ===
    const buscaAberta = centralAssociacaoBuscaAberta.has(chaveWidget);

    if (associacao) {
        const produto = listaProdutosSpData.find(p => p.codigo === associacao.spD
        const linha = produto ? `${produto.codigo} — ${produto.nome} — ${produto.
        return `<div class="central-spdata-linha">
            <span><strong>SP Data:</strong> ${linha}</span>
            <button type="button" class="central-status-toggle" onclick="event.st
        </div>${buscaAberta ? renderBuscaAssociacaoSpData(it, chaveWidget) : ''}`
    }

    return `<div class="central-spdata-linha central-item-sem-desc">
        <span>SP Data: não associado</span>
        <button type="button" class="central-status-toggle" onclick="event.stopPr
    </div>${buscaAberta ? renderBuscaAssociacaoSpData(it, chaveWidget) : ''}`;
}
function toggleBuscaAssociacaoSpData(chaveWidget) {
    if (centralAssociacaoBuscaAberta.has(chaveWidget)) centralAssociacaoBuscaAber
    else centralAssociacaoBuscaAberta.add(chaveWidget);
    // Este widget é reaproveitado tanto na Central do Pedido quanto na
    // Entrada de NF (associação SP Data direto na tela, sem sair pra
    // Cotações) — re-renderiza só o(s) contexto(s) realmente ativo(s).
    if (centralPedidoAtual) renderCentralPedidoCompleto(centralPedidoAtual);
    if (nfeInfoAtual) renderAssociacaoCotacaoXml();
}
// Sugestões iniciais aparecem ao abrir (baseadas no nome oficial do item, se
// houver); busca manual atualiza só a lista de resultados via DOM direto
// (não re-renderiza a Central inteira), senão o campo de texto perderia o
// foco a cada letra digitada.
function renderBuscaAssociacaoSpData(it, chaveWidget) {
    const sugestoesIniciais = it.nomeOficial ? buscarSugestoesSpData(it.nomeOfici
    const listaInicialHTML = sugestoesIniciais.length
        ? sugestoesIniciais.slice(0, 8).map(p => `<div class="central-spdata-opca
        : '<div class="central-item-vazio">Nenhuma sugestão automática — busque m
    return `<div class="central-spdata-busca" onclick="event.stopPropagation()">
        <input type="text" class="form-field" placeholder="Buscar por nome ou cód
```

```
        <div class="central-spdata-opcoes" id="spdata-resultados-${chaveWidget}">
    </div>`;
}
function atualizarBuscaAssociacaoSpData(chaveWidget, codigoItem, texto) {
    const container = document.getElementById(`spdata-resultados-${chaveWidget}`)
    if (!container) return;
    const candidatos = texto.trim() ? buscarSugestoesSpData(texto) : [];
    container.innerHTML = candidatos.length
        ? candidatos.slice(0, 8).map(p => `<div class="central-spdata-opcao" oncl
        : '<div class="central-item-vazio">Nenhum produto encontrado.</div>';
}

function renderLinhaProduto(it, chaveUnica, ctx) {
    const nomeOficialSmartCompras = it.nomeOficial ? upAud(it.nomeOficial) : null
    const observacao = it.descricao && it.descricao !== '---' ? upAud(it.descrica
    const marca = it.fabricante && it.fabricante !== '---' ? it.fabricante : '';
    const expandido = centralProdutosExpandidos.has(chaveUnica);

    // Depois que existe associação confirmada com o SP Data, a identidade
    // interna (SP Data) passa a ser a identificação PRINCIPAL exibida na
    // lista — nada do SmartCompras é apagado, só deixa de ser o título e
    // passa pros detalhes expandidos (ver detalhesHTML abaixo).
    const associacaoSpData = it.codProduto ? listaAssociacoesSpData.find(a => a.c
    const produtoSpData = associacaoSpData ? listaProdutosSpData.find(p => p.codi

    // Relatório de fornecedores ganhadores (fonte complementar à cotação):
    // mostra quantos fornecedores realmente cotaram este item específico —
    // é um fato objetivo, nunca decide recurso/pagamento sozinho.
    const relatorioGanhadoresPedido = centralPedidoAtual ? listaRelatorioGanhador
    const itemGanhadores = relatorioGanhadoresPedido && it.codProduto ? relatorio
    const participantesResumo = itemGanhadores ? ` · ${itemGanhadores.participant
    const participantesDetalheHTML = itemGanhadores
        ? `<div>Fornecedores que cotaram este item: <strong>${itemGanhadores.part
        : '';

    const tituloHTML = nomeOficialSmartCompras
        ? `<div class="central-item-desc-linha">${nomeOficialSmartCompras}</div>`
        : ((nomeProv) => nomeProv
            ? `<div class="central-item-desc-linha">${escRel(nomeProv)} <small cl
            : `<div class="central-item-desc-linha central-item-sem-desc">Nome of
    // SP Data sempre visível, mas como linha SECUNDÁRIA — nunca substitui o
    // nome real da cotação (já foi assim antes e causou associação errada:
    // nomes do SP Data podem vir cortados/parecidos e escondiam o nome real).
    const spDataSubtituloHTML = produtoSpData ? `<div class="central-item-meta">S

    // Recurso confirmado na entrada da NF/XML (Parte 2/4 da correção) — só
```

```
// exibição, nunca uma ação de escolher aqui.
const recursoItem = (centralPedidoAtual && it.codProduto) ? (() => {
    const c = listaCotacoes.find(x => x.pedido === centralPedidoAtual);
    return c && c.recursosPorItem ? c.recursosPorItem[it.codProduto] : null;
})() : null;
const recursoHTML = recursoItem
    ? `<div>Recurso: <strong>${recursoItem}</strong></div>`
    : (it.codProduto ? '<div style="color:var(--text-light);"><em>Recurso: ai

const detalhesHTML = `<div class="central-item-detalhes">
    ${it.codProduto ? `<div>Código SmartCompras: ${it.codProduto}</div>` : ''
    ${nomeOficialSmartCompras ? `<div>Nome oficial SmartCompras: ${nomeOficia
    ${observacao ? `<div>Descrição/observação original SmartCompras: ${observ
    ${marca ? `<div>Marca: ${marca}</div>` : ''}
    ${it.embalagem ? `<div>Embalagem: ${it.embalagem}</div>` : ''}
    ${it.quantidade ? `<div>Quantidade: ${it.quantidade}</div>` : ''}
    ${it.precoUnitario ? `<div>Valor Unitário: R$ ${it.precoUnitario}</div>`
    ${it.precoTotal ? `<div>Valor Total: R$ ${it.precoTotal}</div>` : ''}
    ${recursoHTML}
    ${participantesDetalheHTML}
    ${renderAssociacaoSpDataWidget(it, chaveUnica)}
</div>`;

const resumoMeta = [it.codProduto ? `Cód. ${it.codProduto}` : '', it.quantida

// Fase 27: linha única recolhível — cabeçalho (nome, SP Data, status) e,
// ao abrir, cotação + comparação Cotação × NF × ERP + divergências da
// auditoria do MESMO item. Abre/fecha por classe (sem redesenhar a tela).
const c = ctx || {};
const badges = (c.comp ? (c.comp.divergencias.length && c.comp.resolvido
        ? `<span class="xml-item-badge pronto">✓ Resolvido</span>`
        : c.comp.divergencias.length
        ? `<span class="xml-item-badge pendente">⚠ ${c.comp.divergencias.leng
        : (!c.comp.temNf && c.comp.temErp) ? `<span class="xml-item-badge pro
        : `<span class="xml-item-badge pronto">✓ Confere</span>`) : (c.chegou
    + (c.divsPendentes ? ` <span class="xml-item-badge pendente">Auditoria pe
return `<div class="central-item-linha central-prod${expandido ? ' aberto' :
    <div class="central-prod-cab" onclick="toggleCentralProduto('${chaveUnica
        ${tituloHTML}
        ${spDataSubtituloHTML}
        <div class="central-item-meta"><span>${resumoMeta}${participantesResu
    </div>
    <div class="central-colapso"><div class="central-colapso-inner">
        ${detalhesHTML}
        ${c.compHTML ? `<div class="central-secao-titulo">Comparação Cotação
        ${c.divsHTML ? `<div class="central-secao-titulo">Divergências (audit
```

```
        </div></div>
    </div>`;
}
function toggleCentralProduto(chave, el) {
    const abrir = !centralProdutosExpandidos.has(chave);
    if (abrir) centralProdutosExpandidos.add(chave); else centralProdutosExpandid
    const linha = el && el.closest ? el.closest('.central-item-linha') : null;
    if (linha) linha.classList.toggle('aberto', abrir); // só troca a classe: ani
    else renderCentralPedidoCompleto(centralPedidoAtual);
}

function toggleCentralFornecedor(cnpj, el) {
    const abrir = !centralFornecedoresAbertos.has(cnpj);
    if (abrir) centralFornecedoresAbertos.add(cnpj); else centralFornecedoresAber
    const card = el && el.closest ? el.closest('.central-fornecedor') : null;
    if (card) card.classList.toggle('aberto', abrir);
    else renderCentralPedidoCompleto(centralPedidoAtual);
}
function toggleCentralStatusForm(key) {
    if (centralStatusFormAberto.has(key)) centralStatusFormAberto.delete(key);
    else centralStatusFormAberto.add(key);
    renderCentralPedidoCompleto(centralPedidoAtual);
}

async function confirmarAlteracaoStatusCentral(anotacaoId, ocIdx, divIdx) {
    const key = `${ocIdx}-${divIdx}`;
    const select = document.getElementById(`central-status-select-${key}`);
    const obsInput = document.getElementById(`central-status-obs-${key}`);
    const novoStatus = select.value;
    const observacao = obsInput.value.trim();

    const nota = listaAnotacoes.find(a => a.id === anotacaoId);
    if (!nota) return;
    // Clona profundamente antes de mexer — nunca muta o array em memória
    // direto, só depois que o Firestore confirmar a escrita.
    const ocorrencias = JSON.parse(JSON.stringify(nota.ocorrencias || []));
    const oc = ocorrencias[ocIdx];
    if (!oc || !oc.divergencias || !oc.divergencias[divIdx]) return;
    const d = oc.divergencias[divIdx];
    d.status = novoStatus;
    if (!d.historico) d.historico = [];
    // HISTÓRICO NUNCA É APAGADO — só acrescenta uma entrada nova. O texto
    // gerado/salvo na anotação continua exatamente como estava.
    const tipoEntrada = novoStatus === 'resolvida' ? 'resolucao' : novoStatus ===
    d.historico.push({ tipo: tipoEntrada, data: new Date().toISOString().slice(0,
```

```
    try {
        await anotacoesTextoCollection.doc(anotacaoId).update({ ocorrencias, atua
        toast('✓ Status atualizado.');
        centralStatusFormAberto.delete(key);
        renderCentralPedidoCompleto(centralPedidoAtual);
    } catch (e) {
        console.error('Erro ao atualizar status da divergência:', e);
        toast('× Erro ao atualizar status.');
    }
}

// Origem/Data do Pedido/Data Limite/Observação editáveis direto na Central
// (igual ao Gerenciador de NF: abrir, editar, salvar, sem trocar de tela) —
// grava só esses campos em cotacoes/{pedido}, nunca mexe em fornecedores/
// itens (esses continuam vindo só da importação do XML ou cadastro manual).
async function salvarCabecalhoCotacaoCentral() {
    if (!centralPedidoAtual) return;
    const dados = {
        origem: document.getElementById('central-origem').value,
        dataPedido: document.getElementById('central-data-pedido').value,
        dataLimite: document.getElementById('central-data-limite').value,
        observacao: document.getElementById('central-observacao-cotacao').value.t
        atualizadoEm: new Date().toISOString()
    };
    try {
        await cotacoesCollection.doc(centralPedidoAtual).set(dados, { merge: true
        const nota = listaAnotacoes.find(a => a.pedido === centralPedidoAtual);
        if (nota) {
            const novoTitulo = dados.origem ? `${centralPedidoAtual} - ${dados.or
            if (novoTitulo !== nota.titulo) await anotacoesTextoCollection.doc(no
        }
        toast('✓ Salvo.');
        // Sem isso, o título/dados na tela ficavam desatualizados até a
        // próxima ação que disparasse um re-render — o listener do Firestore
        // não está amarrado a essa tela.
        renderCentralPedidoCompleto(centralPedidoAtual);
    } catch (e) {
        console.error('Erro ao salvar dados do pedido:', e);
        toast('× Erro ao salvar.');
    }
}

function iniciarAuditoriaDoPedidoCentral() {
    const pedido = centralPedidoAtual;
    abrirNovaAuditoria();
    document.getElementById('aud-pedido').value = pedido;
```

```
    buscarCotacaoParaAuditoria();
}

function abrirTextoLivreDoPedidoCentral() {
    const nota = listaAnotacoes.find(a => a.pedido === centralPedidoAtual);
    if (!nota) return toast('Nenhuma anotação encontrada para este pedido ainda.'
    voltarAnotacaoEditorParaCentral = centralPedidoAtual;
    abrirAnotacao(nota.id, true);
}

function voltarParaListaAnotacoes() {
    if (voltarAnotacaoEditorParaCentral) {
        const pedido = voltarAnotacaoEditorParaCentral;
        voltarAnotacaoEditorParaCentral = null;
        abrirCentralPedido(pedido);
        return;
    }
    switchToScreen('screen-anotacoes', 'Anotações');
}

let autoSaveAnotacaoTimeout = null;
function atualizarTituloCabecalhoAnotacao() {   // o título branco do topo acompa
    const h = document.getElementById('main-header-title'), t = document.getEleme
    if (h && t && tela && tela.classList.contains('active')) h.textContent = t.va
}
function agendarAutoSaveAnotacao() {
    atualizarEstadoToolbarAnotacao(); atualizarTituloCabecalhoAnotacao();
    clearTimeout(autoSaveAnotacaoTimeout);
    autoSaveAnotacaoTimeout = setTimeout(() => salvarAnotacaoAtual(false), 1200);
}

async function salvarAnotacaoAtual(mostrarToast) {
    const tituloEl = document.getElementById('anotacao-titulo');
    const corpoEl = document.getElementById('anotacao-corpo');
    if (!tituloEl || !corpoEl) return;

    const titulo = tituloEl.value.trim();
    const conteudo = corpoEl.innerHTML;

    // Nada digitado ainda numa anotação nova: não cria lixo no banco.
    if (!anotacaoAtualId && !titulo && !stripHtmlAnotacao(conteudo)) {
        if (mostrarToast) toast('Nada para salvar.');
        return;
    }

    const dados = {
```

```
        titulo: titulo || 'Sem título',
        conteudo,
        espacamentoLinha: editorEspacamentoLinhaAtual,
        paragrafoCompacto: editorParagrafoCompactoAtual,
        atualizadoEm: new Date().toISOString()
    };

    try {
        if (anotacaoAtualId) {
            await anotacoesTextoCollection.doc(anotacaoAtualId).update(dados);
        } else {
            dados.criadoEm = dados.atualizadoEm;
            const ref = await anotacoesTextoCollection.add(dados);
            anotacaoAtualId = ref.id;
        }
        atualizarTituloCabecalhoAnotacao();
        if (mostrarToast) toast('✓ Anotação salva!');
    } catch (e) {
        console.error('Erro ao salvar anotação:', e);
        if (mostrarToast) toast('× Erro ao salvar anotação.');
    }
}

// Estilo Apple Notas: "✓" no topo conclui (salva e volta) e "⋯" abre as ações da
async function concluirAnotacao() {
    fecharMenuNota(); fecharPopoversRte(); clearTimeout(autoSaveAnotacaoTimeout);
    if (document.activeElement && typeof document.activeElement.blur === 'functio
    await salvarAnotacaoAtual(true);
    voltarParaListaAnotacoes();
}
function fecharMenuNota() { const m = document.getElementById('nota-menu'); if (m
function toggleMenuNota(ev) { if (ev) ev.stopPropagation(); const m = document.ge
function menuNotaAcao(acao) {
    fecharMenuNota();
    if (acao === 'copiar') copiarTextoAnotacao(); else if (acao === 'colar') cola
}
document.addEventListener('click', (e) => { if (!e.target.closest('#nota-menu'))

async function copiarTextoAnotacao() {
    const corpoEl = document.getElementById('anotacao-corpo');
    const texto = corpoEl ? corpoEl.innerText.trim() : '';
    if (!texto) return toast('Nada para copiar.');
    await navigator.clipboard.writeText(texto);
    toast('✓ Texto copiado!');
}
```

```
async function colarTextoAnotacao() {
    const corpoEl = document.getElementById('anotacao-corpo');
    if (!corpoEl) return;
    try {
        const texto = await navigator.clipboard.readText();
        if (!texto) return toast('Área de transferência vazia.');
        corpoEl.focus();
        document.execCommand('insertText', false, texto);
        agendarAutoSaveAnotacao();
        toast('✓ Texto colado!');
    } catch (e) {
        console.error('Erro ao colar:', e);
        toast('× Não foi possível colar. Verifique a permissão de área de transfe
    }
}

function limparTextoAnotacao() {
    const corpoEl = document.getElementById('anotacao-corpo');
    if (!corpoEl || !stripHtmlAnotacao(corpoEl.innerHTML)) return toast('Já está
    showConfirmModal({
        title: 'Limpar Texto',
        message: 'Isso vai apagar todo o texto desta anotação. O título é mantido
        confirmText: 'Limpar',
        confirmClass: 'danger',
        onConfirm: () => {
            corpoEl.innerHTML = '';
            agendarAutoSaveAnotacao();
            toast('✓ Texto limpo.');
        }
    });
}

function excluirAnotacaoAtual() {
    if (!anotacaoAtualId) {
        // anotação nova, ainda não salva - só descarta e volta pra lista sem cri
        clearTimeout(autoSaveAnotacaoTimeout);
        document.getElementById('anotacao-titulo').value = '';
        document.getElementById('anotacao-corpo').innerHTML = '';
        switchToScreen('screen-anotacoes', 'Anotações');
        return;
    }
    showConfirmModal({
        title: 'Excluir Anotação',
        message: 'Deseja excluir esta anotação permanentemente?',
        confirmText: 'Excluir',
        confirmClass: 'danger',
```

```
        onConfirm: async () => {
            try {
                clearTimeout(autoSaveAnotacaoTimeout);
                await anotacoesTextoCollection.doc(anotacaoAtualId).delete();
                anotacaoAtualId = null;
                document.getElementById('anotacao-titulo').value = '';
                document.getElementById('anotacao-corpo').innerHTML = '';
                toast('🗑️ Anotação excluída.');
                switchToScreen('screen-anotacoes', 'Anotações');
            } catch (e) {
                console.error('Erro ao excluir anotação:', e);
                toast('× Erro ao excluir.');
            }
        }
    });
}

// Formatação de texto rico (negrito, itálico, títulos, listas). execCommand
// ainda funciona bem pra esses comandos básicos em todos os navegadores atuais,
// apesar de "deprecated" — é a forma mais simples de dar edição tipo
// Word/Notion sem precisar de uma biblioteca externa pesada.
function formatarTextoAnotacao(comando, valor) {
    document.getElementById('anotacao-corpo').focus();
    document.execCommand(comando, false, valor || null);
    agendarAutoSaveAnotacao();
    atualizarEstadoToolbarAnotacao();
}

// --- Estado visual da toolbar (negrito/itálico/sublinhado/bloco ativos, e
// undo/redo habilitados) — precisa refletir o estado REAL do cursor/seleção
// a cada momento, não um valor fixo marcado uma vez. Atualiza via
// selectionchange (filtrado pro editor de anotações), a cada digitação, e
// logo depois de qualquer comando de formatação disparado pelos botões.
function atualizarEstadoToolbarAnotacao() {
    const editor = document.getElementById('anotacao-corpo');
    if (!editor) return;
    ['bold', 'italic', 'underline'].forEach(cmd => {
        const btn = document.querySelector(`.rte-toolbar button[data-cmd="${cmd}"
        if (!btn) return;
        try { btn.classList.toggle('active', document.queryCommandState(cmd)); }
    });
    let blocoAtual = '';
    try { blocoAtual = (document.queryCommandValue('formatBlock') || '').toUpperC
    document.querySelectorAll('.rte-toolbar button[data-block]').forEach(btn => {
        const alvo = btn.dataset.block;
        const ehParagrafoPadrao = alvo === 'P' && (blocoAtual === '' || blocoAtua
```

```
        btn.classList.toggle('active', blocoAtual === alvo || ehParagrafoPadrao);
    });
    ['undo', 'redo'].forEach(cmd => {
        const btn = document.querySelector(`.rte-toolbar button[data-cmd="${cmd}"
        if (!btn) return;
        try { btn.disabled = !document.queryCommandEnabled(cmd); } catch (e) { /*
    });
}
document.addEventListener('selectionchange', () => {
    const editor = document.getElementById('anotacao-corpo');
    if (!editor) return;
    const sel = window.getSelection();
    if (!sel || !sel.anchorNode || !editor.contains(sel.anchorNode)) return;
    atualizarEstadoToolbarAnotacao();
});

// --- Popovers da barra de ferramentas (cor, tamanho, grade de tabela) ---
// Guarda a seleção de texto feita no editor antes de abrir um popover — clicar
// num botão da toolbar tiraria o foco do editor e perderia a seleção, então
// salvamos o Range aqui (o onmousedown com preventDefault nos botões evita que
// o navegador desfaça a seleção antes mesmo do clique acontecer).
let rteSelecaoSalva = null;
function salvarSelecaoRte() {
    const sel = window.getSelection();
    if (sel && sel.rangeCount > 0 && document.getElementById('anotacao-corpo').co
        rteSelecaoSalva = sel.getRangeAt(0).cloneRange();
    }
}
function restaurarSelecaoRte() {
    const editor = document.getElementById('anotacao-corpo');
    editor.focus();
    if (rteSelecaoSalva) {
        const sel = window.getSelection();
        sel.removeAllRanges();
        sel.addRange(rteSelecaoSalva);
    }
}
function fecharPopoversRte() {
    document.querySelectorAll('.rte-popover').forEach(p => { p.style.display = 'n
}
function toggleRtePopover(id) {
    salvarSelecaoRte();
    const pop = document.getElementById(id);
    const estavaAberto = pop.style.display === 'block';
    fecharPopoversRte();
    // A barra rola para o lado (overflow) e o Safari corta qualquer menu que est
```

```
    if (!pop._wrap) pop._wrap = pop.closest('.rte-popover-wrap');
    if (pop.parentElement !== document.body) { document.body.appendChild(pop); po
    pop.style.display = estavaAberto ? 'none' : 'block';
    if (id === 'rte-table-popover') montarGradeTabela();
    if (id === 'rte-size-popover') atualizarSizePopoverAtivo();
    if (id === 'rte-spacing-popover') atualizarPopoverEspacamentoAtivo();
    if (!estavaAberto) posicionarPopoverDentroDaTela(pop);
}
// Os botões de tamanho/espaçamento/tabela ficam perto da borda direita da
// toolbar (ela tem 2 fileiras de ícones, muitos botões). O popover sempre
// abria alinhado à esquerda do próprio botão (left:0) sem checar se cabia na
// tela — perto da borda direita ele estourava pra fora e ficava flutuando
// sobre o conteúdo, cortado. Agora mede a posição real antes de mostrar e
// alinha pela direita quando não há espaço suficiente à esquerda.
function posicionarPopoverDentroDaTela(pop) {
    // A barra de formatação agora é uma cápsula fixa embaixo (como na Apple), qu
    // dela, centralizado no botão e sem estourar a tela (um popover dentro da ba
    const wrap = pop._wrap || pop.closest('.rte-popover-wrap'), dock = document.g
    const br = wrap.getBoundingClientRect(), dr = dock.getBoundingClientRect();
    pop.style.position = 'fixed'; pop.style.top = 'auto'; pop.style.right = 'auto
    if (window.matchMedia('(min-width:900px)').matches) { pop.style.bottom = 'aut
    else { pop.style.top = 'auto'; pop.style.bottom = Math.round(window.innerHeig
    const w = pop.getBoundingClientRect().width;
    pop.style.left = Math.round(Math.max(8, Math.min(br.left + br.width / 2 - w /
}
document.addEventListener('click', (event) => {
    if (!event.target.closest('.rte-popover-wrap') && !event.target.closest('.rte
});

// --- Cor do texto ---
function aplicarCorTextoAnotacao(cor) {
    restaurarSelecaoRte();
    document.execCommand('foreColor', false, cor || 'inherit');
    fecharPopoversRte();
    agendarAutoSaveAnotacao();
}

// --- Tamanho do texto (em px) ---
// execCommand('fontSize') só aceita valores de 1 a 7 (sem controle em px), então
// envolvemos a seleção manualmente num <span style="font-size:Npx">.
function aplicarTamanhoTextoAnotacao(px) {
    restaurarSelecaoRte();
    const sel = window.getSelection();
    if (!sel || sel.rangeCount === 0 || sel.isCollapsed) { fecharPopoversRte(); r
    const range = sel.getRangeAt(0);
    const span = document.createElement('span');
```

```
    span.style.fontSize = px + 'px';
    try {
        range.surroundContents(span);
    } catch (e) {
        // Seleção atravessa múltiplos elementos (ex: parte de duas linhas) —
        // nesse caso, extrai o conteúdo e envolve manualmente.
        span.appendChild(range.extractContents());
        range.insertNode(span);
    }
    sel.removeAllRanges();
    fecharPopoversRte();
    agendarAutoSaveAnotacao();
}

// Detecta o tamanho de fonte no ponto do cursor/seleção, subindo pela árvore
// do DOM até achar um <span style="font-size:...">. Usado só pra marcar a
// opção certa com um check quando o popover abre — não é fixo, reflete o
// estado real.
function obterTamanhoFonteAtual() {
    const editor = document.getElementById('anotacao-corpo');
    const sel = window.getSelection();
    if (!editor || !sel || !sel.rangeCount) return 16;
    let node = sel.anchorNode;
    if (node && node.nodeType !== 1) node = node.parentNode;
    while (node && node !== editor && editor.contains(node)) {
        if (node.style && node.style.fontSize) {
            const px = parseInt(node.style.fontSize, 10);
            if (!isNaN(px)) return px;
        }
        node = node.parentNode;
    }
    return 16;
}
function atualizarSizePopoverAtivo() {
    const atual = obterTamanhoFonteAtual();
    document.querySelectorAll('.rte-size-option[data-size]').forEach(btn => {
        btn.classList.toggle('active', parseInt(btn.dataset.size, 10) === atual);
    });
}

// --- Espaçamento entre linhas / parágrafos ---
// Aplicado ao documento inteiro (não por parágrafo — mantém a implementação
// simples, como pedido), e persistido como campo próprio da anotação, então
// volta a aparecer do jeito que foi deixado da próxima vez que a nota abrir.
const ESPACAMENTOS_LINHA_ANOTACAO = { simples: '1.35', '115': '1.55', '15': '1.9'
let editorEspacamentoLinhaAtual = '';
```

```
let editorParagrafoCompactoAtual = false;
function aplicarEstadoEspacamentoEditor(valorLineHeight, compacto) {
    const editor = document.getElementById('anotacao-corpo');
    if (!editor) return;
    editor.style.lineHeight = valorLineHeight || '';
    editor.classList.toggle('rte-compacto', !!compacto);
    editorEspacamentoLinhaAtual = valorLineHeight || '';
    editorParagrafoCompactoAtual = !!compacto;
}
function aplicarEspacamentoLinhaAnotacao(chave) {
    const valor = ESPACAMENTOS_LINHA_ANOTACAO[chave] || '';
    const editor = document.getElementById('anotacao-corpo');
    if (editor) editor.style.lineHeight = valor;
    editorEspacamentoLinhaAtual = valor;
    atualizarPopoverEspacamentoAtivo();
    fecharPopoversRte();
    agendarAutoSaveAnotacao();
}
function alternarParagrafoCompactoAnotacao() {
    const editor = document.getElementById('anotacao-corpo');
    if (!editor) return;
    editorParagrafoCompactoAtual = !editorParagrafoCompactoAtual;
    editor.classList.toggle('rte-compacto', editorParagrafoCompactoAtual);
    atualizarPopoverEspacamentoAtivo();
    agendarAutoSaveAnotacao();
}
function atualizarPopoverEspacamentoAtivo() {
    document.querySelectorAll('.rte-size-option[data-spacing]').forEach(btn => {
        btn.classList.toggle('active', (ESPACAMENTOS_LINHA_ANOTACAO[btn.dataset.s
    });
    const btnCompacto = document.getElementById('rte-btn-compacto');
    if (btnCompacto) btnCompacto.classList.toggle('active', editorParagrafoCompac
}

// --- Tabela: grade visual estilo Word/Google Docs, em vez de prompt() ---
function montarGradeTabela() {
    const grid = document.getElementById('rte-table-grid');
    const label = document.getElementById('rte-table-grid-label');
    if (grid.childElementCount > 0) return; // já montada, só reaproveita
    for (let l = 1; l <= 6; l++) {
        for (let c = 1; c <= 6; c++) {
            const cell = document.createElement('div');
            cell.dataset.linhas = l;
            cell.dataset.colunas = c;
            cell.onmouseenter = () => destacarGradeTabela(l, c);
            cell.onclick = () => { inserirTabelaAnotacao(l, c); };
```

```
            grid.appendChild(cell);
        }
    }
    grid.onmouseleave = () => { label.textContent = 'Selecione o tamanho'; destac
}
function destacarGradeTabela(linhas, colunas) {
    document.getElementById('rte-table-grid-label').textContent = linhas && colun
    document.querySelectorAll('#rte-table-grid div').forEach(cell => {
        const l = parseInt(cell.dataset.linhas), c = parseInt(cell.dataset.coluna
        cell.classList.toggle('rte-cell-hover', l <= linhas && c <= colunas);
    });
}

// Insere uma tabela editável no ponto do cursor — permite montar ou colar um
// pedaço de planilha (ao copiar células do Excel/Google Sheets e colar aqui
// dentro, o navegador já preserva a tabela automaticamente também).
function inserirTabelaAnotacao(linhas, colunas) {
    if (!linhas || !colunas) return;
    restaurarSelecaoRte();

    let html = '<table class="anotacao-tabela"><tbody>';
    for (let i = 0; i < linhas; i++) {
        html += '<tr>';
        for (let j = 0; j < colunas; j++) {
            html += '<td> </td>';
        }
        html += '</tr>';
    }
    html += '</tbody></table><p><br></p>';

    document.execCommand('insertHTML', false, html);
    fecharPopoversRte();
    agendarAutoSaveAnotacao();
}
```